

Affordable Mistakes: Severity-Aware Multiparty Session Types for Participants that Choose Wrongly*

Anonymous author

Anonymous affiliation

Anonymous author

Anonymous affiliation

Abstract

Multiparty session types guarantee that well-typed participants interact correctly, on the hypothesis that each participant *is* its type. A participant that was instructed rather than compiled — a language-model agent — breaks the hypothesis in one specific way: at a branch where it should select *A* it may select *B*. Forbidding this is impossible, and would be useless if it were possible. We change what the discipline is for. It does not prevent wrong choices; it prevents *catastrophic* ones and reports the rest as risk. The fault model is the participant’s own selection at a branch, defined through the guards that asserted global types already carry. The consequence of a wrong selection is classified by *outcome* against a STRIPS-style world: goal still reachable, goal lost but nothing harmed, or a hazard reachable through a *point of no return*. Cascading is a budget *k* of misselections, with no probabilities anywhere. The contribution is a *well-formedness condition* on global types, *k*-misselection tolerance, with the standard MPST payoff: a session typed against a *k*-tolerant protocol by the base discipline’s direct-conformance judgment is hazard-free on every run with at most *k* misselections, and budgets distribute over participants. The condition is exactly characterized by one syntax-directed rule, composes modularly across sequential composition, and is decidable for regular protocols: the unfolding closure of a μ -type is proved finite, and an executable decision procedure is verified on it. The bridge holds for recursive sessions, and the four repairs — guard, reorder, narrow, compensate — are proved sound. A bystander — a role acting before its turn in an ungated deployment — preserves the condition under a permutation relation whose semantic side conditions a syntactic variable-disjointness check discharges, and the tool’s decision on the boolean fragment is a kernel extracted from the proof. All of this is mechanized in Coq, axiom-free. An implementation over an existing achievability compiler shows that two existing corpora contain no decision points or irreversible tools at all — the question is invisible to them — and, on a purpose-built benchmark of seventeen protocols, classifies 55 branch verdicts, finds nine 0-tolerant protocols each naming its point-of-no-return action, verifies three repairs, and demonstrates that modular checking with the interface abstraction the theory licenses is linear where whole-system analysis is exponential. A live-agent experiment — 340 runs of two production language models, with and without a requester’s pressure to take shortcuts — finds every observed catastrophe consistent with the theorem, the catastrophe rate ordered by tolerance degree, and six of the nineteen *Catastrophic* verdicts taken by a real agent. On 162 real skills from thirteen public repositories and four authored specification pairs, run by the same agents in a shell and a file-only runtime with every artifact verified by recomputation, the checker’s refusals coincide with the runs that ended in wasted tokens, honest failure or a fabricated result: at realistic input sizes no refuted run produced a correct artifact, at 5.1 M tokens spent to produce none. The check itself costs no model tokens on the refutations and 31.2% of one agent run where the document needs semantic compaction.

2012 ACM Subject Classification Theory of computation → Type structures; Theory of computation → Process calculi; Software and its engineering → Software verification

Keywords and phrases Session types, Multiparty, Behavioural types, Safety, Irreversibility, Fault tolerance, Mechanized metatheory

* **WORKING DRAFT** — not for submission. Base rules follow `paper/tas.tex`, authoritative for Section 3.



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

Digital Object Identifier 10.4230/LIPIcs.ECOOP..0

Supplementary Material [Anonymous supplementary material](#)

Funding [Anonymous funding](#)

Acknowledgements [Anonymous acknowledgements](#)

WIP: Status. Everything in Sections 5–9 is mechanized (**proof/**, 80 results, all **Print Assumptions** closed). The evaluation numbers in Section 10 are real (**scripts/severity_eval.py**, results in **paper/WIP/results/**). Scope caveats are stated where they apply. The predecessor draft was withdrawn after its own mechanized audit found it vacuous (Section 13).

1 Introduction

Multiparty session types (MPST) guarantee communication safety, session fidelity and deadlock-freedom for systems whose participants *are* their local types [1]: the code was checked, so every run is a compliant run. A participant that was instructed rather than compiled breaks that hypothesis in a specific way. At a choice point it may take branch B where the situation demanded branch A — not by violating the protocol (both branches are in the type) but by judging the situation wrongly. This is the ordinary condition of a language-model agent occupying a role, and it is why such agents are employed: their latitude.

The tempting response is to make the type system prevent it. That is doubly wrong: impossible, because the judgement is the participant's, not the protocol's; and undesirable, because a participant that may never choose wrongly is one that may never choose. What is needed is *triage*.

The reframing. Do not ask whether the participant complied. Ask *what a wrong choice costs*. Three answers matter, and the middle one is what existing disciplines cannot express (Figure 1):

- Benign — a detour; the goal is still reachable.
- Futile — the goal is lost, but nothing is harmed. *Failure is not disaster*.
- Catastrophic — a hazard becomes reachable.

A system that blocks Futile is useless; one that permits Catastrophic is dangerous. Separating them is the whole job. Thesis: *session types say what may happen; this says what you can survive*.

What is and is not new. The ingredients are old; the discipline is not. Branch guards on global types are design-by-contract MPST [2, 4, 5], and guard violation on selection is the alarm condition of the monitoring line [3]. Whether a goal remains reachable is dead-end detection in planning [29, 30]; whether an effect can be undone is undoability checking [31, 32]. Bounded fault tolerance is classical in planning [27, 28] and in MPST with crash-stop failures [8, 9]. A syntax-directed system equivalent to a model checker is a known template [21, 22]. We claim none of these. We claim one thing:

A well-formedness condition on global types — k -misselection tolerance — whose fault model is the participant's own selection at a branch, whose analytic payload is a classification of each wrong selection by outcome against a world, and which delivers the standard MPST guarantee: a session typed against a k -tolerant protocol is

hazard-free within budget k , with budgets distributing over participants. The condition is exactly characterized, modular, decidable for regular protocols, and mechanized end to end.

We are careful with the word *type system*. The condition is on global types, as well-assertedness [2] and deadlock-freedom conditions are; the participants are typed by the base discipline’s direct-conformance judgment, unchanged. The type-theoretic content is the theorem that couples the two (Section 7), not the rule that decides the condition.

Contributions.

- C1** *Misselection and severity* (§4, §5): guard failure as the fault model; a budgeted semantics; the Benign/Futile/Catastrophic partition; the point of no return.
- C2** *Exact characterization* (§6): one rule, T-CHOICE-SAFE, sound and complete for k -bounded hazard-freedom, so a failure is a genuine risk (✓ TC_exact).
- C3** *From protocols to programs* (§7): a session typed against a k -tolerant protocol is hazard-free on every run of cost at most k (✓ bridge_run); per-role allowances summing to k are jointly tolerated (✓ budget_distributes); the bridge holds for recursive sessions and protocols (✓ bridge_mu); and it survives bystander interleavings under semantic independence conditions that a syntactic STRIPS check discharges (✓ bridge_interleaved, ✓ strips_swap_act).
- C4** *Regular protocols* (§8): μ -types with unfolding as substitution; the unfolding closure is finite and step-closed (✓ cands_closed); the finite fragment embeds, so the rule is exactly non-reachability in a product graph (✓ TC_regular); the bound on loop re-entry; and an executable decision procedure for μ -types proved sound and complete (✓ decide_mu_correct).
- C5** *Repairs, modularity and evaluation* (§11, §9, §10): guard, reorder, narrow and compensate are proved sound, the first two exactly (✓ repair_guard_exact, ✓ repair_reorder_pnr); sequential composition is typed against an interface (✓ TC_seq_interface); an implementation, a severity benchmark and a live-agent experiment show that the interface abstraction is what makes modular checking pay, that existing corpora cannot even pose the question, and that the Catastrophic verdicts name branches real agents take; on the boolean fragment the tool’s verdict is cross-checked against a kernel extracted from the verified decision procedure (✓ kernel_first_spec); and on 162 real skills in two runtimes the achievability verdicts predict which agent runs are wasted or wrong, with the token cost of the check measured against the token cost of the runs it replaces.

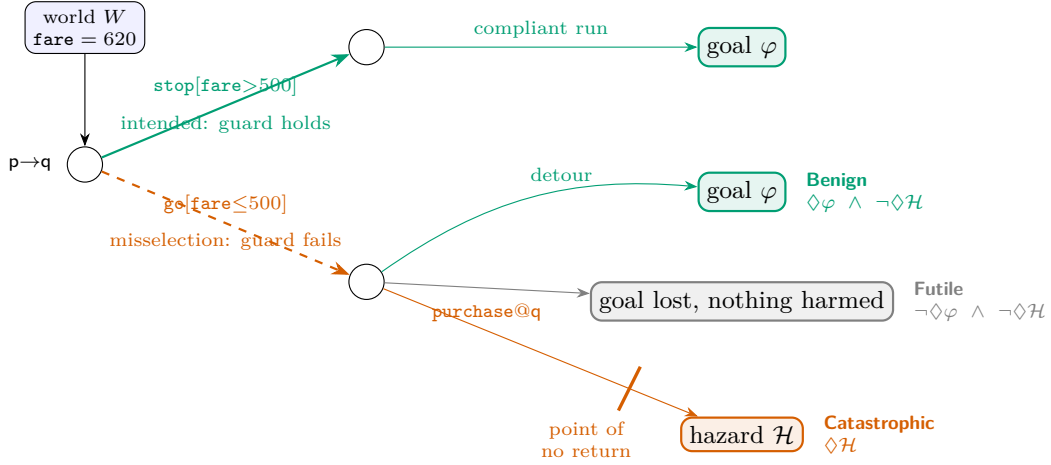
A design expectation was refuted by the mechanization and a benchmark finding refines it: tolerance is anti-monotone in the capability context for a fixed hazard (✓ tolerance_antitone_in_ctx), yet a new tool that re-establishes a lost atom can *remove* a point of no return (§10).

2 Overview

Running example. A planner p and a worker q holding a purchase tool. Atoms **verified**, **booked**, **funds**; **verify** adds the first, **purchase** adds the second and deletes the third, and no tool restores **funds**. The protocol offers two branches:

$$G_{\text{bad}} = p \rightarrow q : \{ \text{safe. verify}@q. \text{purchase}@q. \text{End} , \text{fast. purchase}@q. \text{End} \}$$

with goal $\varphi = \text{booked} \wedge \text{verified}$. Under the default instantiation (§4) **safe** is intended because the goal survives it and **fast** is a misselection because it does not. The tool reports, and



■ **Figure 1** The idea. At a guarded choice in world W , the branch whose guard holds is *intended*; taking another is a *misselection*. Its severity is a property of the residual: **Benign** if the goal is still reachable, **Futile** if the goal is lost but no hazard is reachable, **Catastrophic** if a hazard is reachable — typically by crossing a *point of no return*, an irreversible capability after which the goal cannot be recovered.

the Coq development proves for the same instance: G_{bad} is 0-tolerant ($\checkmark G_{\text{bad_is_0_tolerant}}$) — if the participant never misselects, nothing bad happens, which is where classical MPST stops; it is *not* 1-tolerant ($\checkmark G_{\text{bad_not_1_tolerant}}$) — one wrong choice purchases unverified, and *purchase* is the point of no return; and narrowing away *fast* is tolerant at every k ($\checkmark G_{\text{good_is_k_tolerant}}$). The headline number is the *tolerance degree* k^* , the largest k the protocol survives.

For a protocol with two decision points the tool’s output is what an engineer needs (Section 10, *migration_backup*):

```
migration_backup: tolerance degree k* = 1
choice nodes 4, branches 5, irreversible ['drop_old']
/choice@ops#0
  backup           Benign      intended
  skip_backup      Catastrophic misselection PNR=drop_old
/choice@ops#0/backup/choice@ops#2
  drop_old         Benign      intended
  keep_old         Benign      intended
/choice@ops#0/skip_backup/choice@ops#1
  drop_old         Catastrophic misselection PNR=drop_old
first catastrophe within budget 2:
  choose/skip_backup > act/migrate > choose/drop_old > act/drop_old
repair (narrow):
  remove skip_backup at /choice@ops#0
  remove drop_old at /choice@ops#0/skip_backup/choice@ops#1
```

“Tolerant of one wrong choice; two wrong choices reach data loss, here is the path and the action that burns the bridge” is actionable. “Impossible” is not.

Architecture. Figure 2: a compile-time check on the pack, before any participant runs, with no language model in the trusted core — the base discipline’s trust boundary. The runtime gate is inherited unchanged: it refuses *out-of-set* attempts (labels the protocol does not offer) before delivery, so those change no state. The deviation this paper analyses is *in-set*: a branch the type permits and the situation forbids.

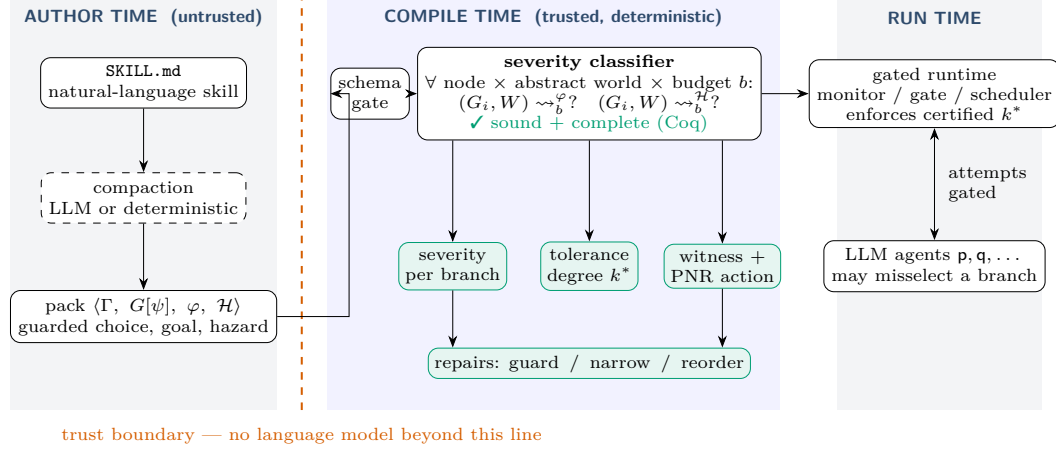


Figure 2 Architecture. Compaction from prose to a pack is untrusted and may use a language model; nothing beyond the trust boundary does. The severity classifier runs the discipline’s existing reachability search pointwise at every branch of every choice, at every abstract world and budget, and emits a per-branch severity, the tolerance degree k^* , a witness naming the point-of-no-return action, and repair suggestions. The certified pack drives the gated runtime that refuses out-of-protocol attempts.

3 The Base Discipline

We build on a goal-marked, capability-guarded synchronous multiparty discipline; `tas.tex` is authoritative and this section fixes notation. Predicates $p \in \text{Pred}$, numeric variables $x \in \text{Var}$, roles p, q , capabilities $a \in \text{Cap}$, labels $\ell \in \text{Lab}$. The state logic is quantifier-free linear integer arithmetic with uninterpreted booleans; a world $W = \langle B, N \rangle$ evaluates it. A capability context maps each possessed tool to a guarded STRIPS effect $\Gamma(a) = \langle pre_a, eff_a \rangle$, $eff_a = \langle Add_a, Del_a, Asg_a, Nd_a \rangle$. Firing is WORLD-ACT: $\langle W \rangle a \langle W' \rangle$ iff $W \models pre_a$ and $W' \in \text{apply}(eff_a, W)$. Global types are coinductive regular trees

$$G ::=^{coind} p \rightarrow q : \{ \ell_i. G_i \}_{i \in I} \mid a @ p. G \mid \checkmark \varphi. G \mid \text{End}$$

and sessions $\mathbb{M} = p_1[P_1] \parallel \dots$ of processes $P ::=^{coind} \mathbf{0} \mid p! \{ \ell_i. P_i \} \mid p? \{ \ell_i. P_i \} \mid a. P$ are typed *directly* against G by the coinductive judgment $G \vdash (\mathbb{M}; W)$ — no local types, projection, or separate subtyping [16, 17] — whose T-COMM requires the sender to offer exactly the protocol’s labels, the receiver at least them, and every branch to check against the same residual. Achievability is existential may-reachability of a goal-satisfying configuration. The base metatheory (subject reduction, session fidelity, refutation soundness) is mechanized for the finite fragment. This paper adds one annotation to choice, one counter to the semantics, and one well-formedness condition.

4 Misselection and the Budget

► **Definition 1** (Guarded choice). A choice node carries, per branch, the predicate that makes that branch the intended one: $p \rightarrow q : \{ \ell_i[\psi_i]. G_i \}_{i \in I}$. In world W , branch i is intended if $W \models \psi_i$; taking a branch with $W \not\models \psi_i$ is a misselection.

The syntax is that of asserted global types [2], whose well-formedness conditions guarantee that a compliant participant can always pick a satisfied branch. We keep the syntax and

drop the guarantee: the guards are not a constraint on an implementation but the *definition of wrong*.

Default instantiation. Guards need not be written. When a branch carries none, the implementation uses the *rational-choice* default: a branch is intended in W iff the goal is still reachable from its residual. Under this default a misselection is a choice that forfeits the goal, and once the goal is forfeit every subsequent choice is a misselection — there is no longer a right one — so the budget counts decisions taken after the point at which the task was lost. Explicit guards override the default where the intended branch is a matter of policy rather than reachability (Section 10 uses both).

► **Definition 2** (Budgeted reachability). $(G, W) \rightsquigarrow_b^{\mathcal{H}}$ holds when a hazard state is reachable from (G, W) using at most b misselections (Coq: `reach_haz`): the head rules of the base semantics, plus

$$\begin{array}{c} \text{RH-COMM-OK} \frac{\ell_i[\psi_i].G_i \in \text{brs} \quad W \models \psi_i \quad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\text{p} \rightarrow \text{q} : \text{brs}, W) \rightsquigarrow_b^{\mathcal{H}}} \\[10pt] \text{RH-COMM-DEV} \frac{\ell_i[\psi_i].G_i \in \text{brs} \quad W \not\models \psi_i \quad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\text{p} \rightarrow \text{q} : \text{brs}, W) \rightsquigarrow_{b+1}^{\mathcal{H}}} \end{array}$$

Compliant progress is free; a misselection costs one unit. Choices marked as controlled by the environment are resolved demonically at no cost.

► **Definition 3** (k -misselection tolerance). G is k -misselection-tolerant for hazard $\mathcal{H}$ under Γ from W_0 if $\neg(G, W_0) \rightsquigarrow_k^{\mathcal{H}}$.¹ 0-tolerance is classical MPST safety.

The budget is possibilistic: a design parameter of the deployment, not a measurement of the participant. Every theorem below holds for every participant that misselects at most k times, however it does so.

5 Severity and the Point of No Return

Let φ be the goal and $\mathcal{H}$ the hazard; $\rightsquigarrow_b^{\varphi}$ is budgeted goal reachability.

► **Definition 4** (Severity). For a residual (G, W) at budget b : *Catastrophic* if $(G, W) \rightsquigarrow_b^{\mathcal{H}}$; otherwise *Futile* if $\neg(G, W) \rightsquigarrow_b^{\varphi}$; otherwise *Benign*.

► **Theorem 5** (Partition). The classes are pairwise disjoint, and exhaustive whenever the two reachability questions are decidable. ✓ `severity_disjoint`, ✓ `severity_exhaustive`

The content is the separation of Futile from Catastrophic. In model-based safety assessment [35] severity is supplied by the analyst; here it is computed from goal reachability.

► **Definition 6** (Point of no return). (G, W) is a point of no return for φ if $\neg(G, W) \rightsquigarrow_b^{\varphi}$. An action is goal-destroying at (G, W) if the goal is reachable before it fires and at no successor.

¹ We avoid two established names. In MPST, k -safety is a buffer bound [14]; in hyperproperties it is a property of k traces [15]. k -resilient plans survive k action failures [27]; our fault is the participant's own selection and our target is a hazard, not the goal.

Default hazard. Because effects are STRIPS, an atom in Del_a is restored only by some b with the atom in Add_b ; if Γ has no such b , a is *irreversible*. The implementation's default hazard, needing no annotation, is *burning a bridge while lost*: an irreversible action fires from a configuration after which the goal is unreachable. Tools whose real-world effect cannot be undone but whose pack models no delete-list (an email cannot be unsent) are declared irreversible explicitly. This is dead-end detection [29] and undoability checking [31] lifted to a protocol; unrestricted reversibility checking is harder than planning [32], and our decidability rests on the widened finite-range fragment the checker already commits to.

6 The Well-Formedness Condition, Exactly

$\vdash_b G, W$ reads: from (G, W) , no hazard arises under any run with at most b misselections.

$$\begin{array}{c}
\text{T-END} \frac{W \not\models \mathcal{H}}{\vdash_b \text{End}, W} \qquad \text{T-GOAL} \frac{W \not\models \mathcal{H} \quad \vdash_b G, W}{\vdash_b \checkmark \varphi.G, W} \\
\\
\text{T-ACT} \frac{W \not\models \mathcal{H} \quad \forall W'. \langle W \rangle a \langle W' \rangle \implies \vdash_b G, W'}{\vdash_b a@\mathbf{p}.G, W} \\
\\
\text{T-CHOICE-SAFE} \frac{\begin{array}{c} W \not\models \mathcal{H} \\ \forall i \in I. W \models \psi_i \implies \vdash_b G_i, W \\ \forall i \in I. W \not\models \psi_i \implies \forall c. b = c+1 \implies \vdash_c G_i, W \end{array}}{\vdash_b \mathbf{p} \rightarrow \mathbf{q} : \{\ell_i[\psi_i].G_i\}_{i \in I}, W}
\end{array}$$

An intended branch is checked at the same budget; a misselectable branch at one less, and not at all when the budget is exhausted. This is “affordable mistake” as a rule.

► **Theorem 7** (Exact characterization). $\vdash_k G, W \iff \neg(G, W) \rightsquigarrow_k^{\mathcal{H}}$, hence a branch is *Catastrophic* iff its residual fails the condition. $\checkmark TC_sound$, $\checkmark TC_complete$, $\checkmark TC_exact$

We do not present this as deep — the rules are the reachability relation with the negation pushed through, and the proof is short. Its value is that a failure is never an artifact of conservative rules, which preserves the refutation asymmetry the discipline is built on, and that the condition is syntax-directed, which is what §9 exploits. Two monotonicity facts complete the picture: tolerance is downward closed in k ($\checkmark tolerance_downward_closed$), and reachability is monotone in Γ , so for a *fixed* hazard granting a tool can only lower k^* ($\checkmark tolerance_antitone_in_ctx$) — the formal case for least privilege, and a refutation of our own prior expectation that more tools mean more recovery.

Not resource-aware typing. The budget resembles the potential of resource-aware session types [23, 24] and graded modalities [25]; the resemblance misleads. Potential is consumed by the program's own execution and exhaustion is a type error. The budget is consumed by an *adversarial* choice the program does not make, so the rule quantifies over deviations rather than amortising cost, and exhaustion is unconstrained.

7 From Protocols to Programs

Everything so far concerns the global type's semantics. The theorem that makes it a discipline about *programs* couples the condition to the base judgment $G \vdash (\mathbf{M}; W)$, lifted to guarded

choice without touching the guards: T-COMM still requires the sender to offer exactly the protocol's labels and every branch to check against the same residual; which label the participant eventually picks is its own business.

► **Definition 8** (Instrumented head-move semantics). *A session step carries the acting role and its cost: 0 for an action or an intended selection, 1 when the sender picks a label whose guard fails in the current world (Coq: `hstep`). A run records the trace of (role, cost) events; total sums costs and `percostr` sums those of role r .*

► **Theorem 9** (Bridge). *If $G \vdash (\mathbb{M}; W)$ and $\vdash_k G, W$, then every instrumented head step of cost $c \leq$ the remaining budget preserves typing and debits the budget by exactly c (`✓ bridge_step`); consequently, along every run of total cost at most k , no configuration is a hazard state (`✓ bridge_run`).*

► **Theorem 10** (Budgets distribute over participants). *Give each role r an allowance k_r . If the allowances of the roles that act sum to at most k , a session typed against a k -tolerant protocol is hazard-free on every run in which each role stays within its own allowance. `✓ budget_distributes`*

Theorem 10 is the cross-participant composition result: the global budget is a contract that can be split among participants and checked per participant.

Recursive sessions. The finite fragment is not the scope of the bridge. Section 8 gives recursive global types $\mu X.G$ their semantics by unfolding; processes carry μ likewise, and a process *unfolds to a head* (deterministically) before it is inspected, so a non-contractive process types against nothing. The conformance judgment becomes coinductive, with one added rule that unfolds the protocol's μ (Coq: `ctypes`, `hstep` in `Mu.v`).

► **Theorem 11** (Bridge, recursive). *An instrumented head step of a typed session is a protocol path of the same misselection cost, and typing is preserved (`✓ sim_step`). Hence a session typed against a recursive protocol on which no hazard is reachable within budget k is hazard-free on every run of cost at most k (`✓ bridge_mu`). On the finite fragment this recovers Theorem 9 through Theorem 18 (`✓ bridge_finite_via_mu`).*

Bystanders. The head-move semantics is the gate's: an out-of-turn attempt is refused. An ungated deployment lets a *bystander* — a role the head does not involve — fire its next capability early. Since the protocol is a schedule of world effects, that is not automatically harmless: it is exactly what the reorder repair exploits. We give the permutation relation and the conditions under which it is (Coq: `Interleave.v`). A *swap* exchanges two adjacent independent nodes, in either direction and under any context: a bystander action $a@r$ and another role's action b ; $a@r$ and a choice between two other roles (when every branch begins with $a@r$); $a@r$ and a goal marker; and two communications $p \rightarrow q$ and $r \rightarrow s$ between disjoint role pairs, each branch of the first continuing with the second. Communications change no world, so their permutation needs no side condition; an action's does: a is *hazard-neutral* (never changes whether the world is a hazard), a and b *commute* (one order re-serializes as the other with the same end world) and preserve each other's enabledness, and a *preserves* the guards and goal markers it passes.

► **Theorem 12** (Bystanders). *Safe swaps preserve the condition at every budget (`✓ swap_safe`) and preserve typing — subject reduction under permutation (`✓ swap_ctypes`). Hence a typed session of a k -tolerant protocol is hazard-free on every interleaved run of cost at most*

k , where before each step the protocol may be permuted by any sequence of safe swaps ($\checkmark$ `bridge_interleaved`).

► **Theorem 13** (Variable disjointness suffices). *For STRIPS capabilities stated extensionally — a precondition supported on a variable set, add and delete lists — if the effects of each action are disjoint from the other’s footprint (its support and effects), the actions commute and preserve each other’s enabledness ($\checkmark$ `strips_commute`, $\checkmark$ `strips_enables`); an action whose effects are disjoint from the support of a predicate preserves it ($\checkmark$ `strips_preserves`, $\checkmark$ `strips_neutral`). Together these discharge every side condition of an action swap ($\checkmark$ `strips_swap_act`).*

Theorem 13 is what the tool checks: for every capability action and every earlier node of another role it could pass, it compares footprints — taking the goal’s and every precondition’s atoms as the support of the derived hazard and of the rational guards — and reports either that the head order is exact or the pairs the gate must serialize. The discipline is synchronous by design (the gate delivers a label when it is taken); what the theorem does not cover is asynchronous buffering, where a sent label waits in a queue, which changes the semantics rather than merely permuting it.

8 Regular Protocols

A regular global type unfolds to a finite graph of protocol nodes; over the finite abstract world of the widened fragment, budgeted reachability is reachability in the product $\text{Node} \times \text{World} \times \{0..k\}$. The development has two layers: an abstract one over any node and world types with computable successors (Coq: `Regular.v`), and its instantiation to μ -types, where the finiteness of the node set is a theorem rather than an assumption (`Mu.v`).

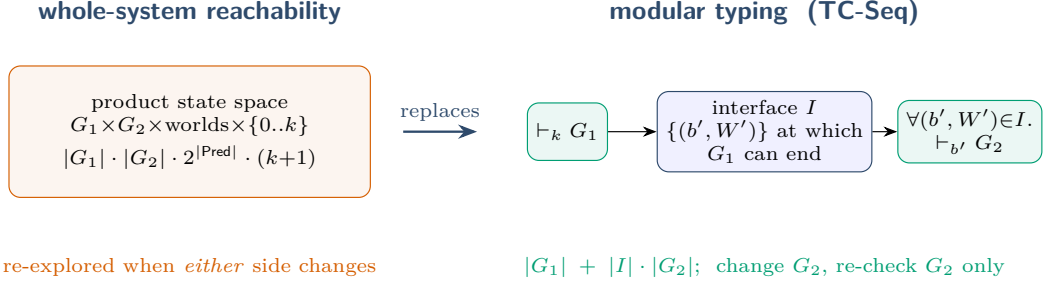
► **Theorem 14** (Product). $(n, w) \rightsquigarrow_b^H$ iff a hazard state is reachable from (n, w, b) in the product graph, whose misselection edges decrement the third component ($\checkmark$ `product_correspondence`).

► **Theorem 15** (Loops meet the budget). *The budget component never increases along a product path ($\checkmark$ `budget_never_increases`); a path with d misselection edges from budget b has $d \leq b$, and exactly $b - b'$ if it ends at budget b' ($\checkmark$ `dev_edges_bounded`, $\checkmark$ `dev_edges_exact`). A loop containing a misselectable branch is therefore traversed wrongly at most k times within budget k .*

► **Theorem 16** (Decidability). *For a finite world type and a finite, successor-closed list of nodes with computable successors, the boolean `decide_reachb` decides $(n, w) \rightsquigarrow_k^H$ from any listed node: it is sound ($\checkmark$ `decide_sound`) and complete ($\checkmark$ `decide_complete`), the latter via a pigeonhole argument that reachability is witnessed by a path no longer than the reachable-state count ($\checkmark$ `reach_bounded_path`); together, $\checkmark$ `decide_reachb_correct`.*

From μ -types to the graph. Regular global types carry $\mu X.G$ and X (de Bruijn indices; Coq: `Gr`). Unfolding is substitution of the closed μ -term for its variable, and the protocol steps of §4 gain one silent step, $\mu X.G \rightarrow G[\mu X.G/X]$. Budgeted reachability on μ -types is the product of Theorem 14 over these steps (`reach_mu`).

► **Theorem 17** (The unfolding closure is finite). *For a closed regular type G_0 there is a computable list $\text{cands}(G_0)$ — the closures of G_0 ’s subterms under their enclosing binders — that contains G_0 and is closed under every protocol step ($\checkmark$ `cands_closed`); every state*



■ **Figure 3** Modularity. A whole-system check of $G_1; G_2$ explores the product of both protocols with the world and the budget and re-explores it when either changes. TC-SEQ types G_2 only at the interface G_1 exposes; the interface form lets that interface be a declared invariant coarser than the concrete exit set.

reachable from G_0 in the product lies in it ($\checkmark$ *reach_in_cands*). The heart is a substitution lemma: substituting the closed μ -term for the variable it binds, after closing the outer context, is closing the extended context ($\checkmark$ *subst_comp*, $\checkmark$ *unfold_close*).

► **Theorem 18** (Embedding). *The finite fragment embeds: $\sim_b^H$ of §4 coincides with *reach_mu* on the image ($\checkmark$ *reach_embed*), so T-CHOICE-SAFE is exactly non-reachability of a hazard in the product graph of the embedded protocol ($\checkmark$ *TC_regular*).*

► **Theorem 19** (Decidability for μ -types). *Over a finite abstract world with decidable guards and computable tool successors, *decide_mu* decides $(G_0, w) \sim_k^H$ for every closed regular G_0 : it runs Theorem 16's procedure on *cands*(G_0), which Theorem 17 shows is successor-closed ($\checkmark$ *decide_mu_correct*).*

► **Proposition 20** (Tolerance degree). *k^* is computable. Whether $k^* = \infty$ is plain reachability in $\text{Node} \times \text{World}$ with misselection edges made free; otherwise k^* is at most the number of misselection edges on a shortest hazard path, bounded by the product size, and is found by iterating Theorem 16. With $|\text{World}| = 2^{|\text{Pred}|}$ the decision problem is in PSPACE; each iteration is linear in the product.*

Proof. Paper proof from Theorems 14–16: dropping the budget component yields the ∞ case; *dev_edges_exact* bounds k^* ; reachability in a succinctly presented graph is in PSPACE. ◀

9 Why a Discipline, and Not a Model Checker

Theorem 7 invites the objection that the rules are a decision procedure in disguise. The objection is old and Naik and Palsberg [21] gave the standard reply. We rest instead on a theorem and an experiment.

► **Theorem 21** (Modular composition). *If $\vdash_k G_1, W$ and $\vdash_{b'} G_2, W'$ for every (b', W') at which G_1 can end from budget k , then $\vdash_k G_1; G_2, W$ ($\checkmark$ *TC_seq*). In interface form: for any I containing those exit points, it suffices that $\vdash_{b'} G_2, W'$ for all $(b', W') \in I$ ($\checkmark$ *TC_seq_interface*).*

The theorem is what a whole-system reachability run cannot be: k becomes a contract on a boundary. But it does not, by itself, say the boundary is small, and the evaluation shows it is not: the *concrete* exit set grows with the number of distinguishable worlds, exponentially when every segment leaves its own atoms behind. What makes modular

checking pay is the *interface* form with a coarser I — and the principled coarsening is the cone of influence, projecting exit worlds onto the atoms the remaining segments read, sound because dropped atoms cannot affect them. With that abstraction the modular check is linear where whole-system analysis is exponential (Section 10, Table 2). This is the axis on which MPST is currently being argued [18]; the community has accepted parameterising typing by a model-checked property [8, 9], and the theorem plus the projection is our concrete instance of that move.

10 Implementation and Evaluation

`skillc severity` implements the analysis over the existing achievability compiler: the checker’s own symbolic search (predicates concrete, numerics symbolic, widened at loop heads) is run pointwise at every branch of every choice, at every abstract world and budget up to $k_{\max} = 4$, with the default guards and hazard of §4–§5 and explicit overrides where declared. It reports per-branch severity, k^* , the first hazard witness, the point-of-no-return action, the narrowing repair, and the bystander pairs the gate must serialize. Eleven tests check the tool’s verdicts on the paper’s instances against the Coq development’s.

Verified kernel. The tool’s search is the graph form of Theorem 17, with widening at loop heads over the arithmetic state; it is not extracted from the proof. To close that gap on the fragment where the proof applies we built a kernel (Coq: `Kernel.v`) that instantiates `decide_mu` on bit-vector worlds with STRIPS actions, guards as world sets, and the derived hazard as a bit that an irreversible action sets whenever the goal is no longer reachable from its continuation. The decision — the least budget at which a hazard is reachable — is proved correct ($\checkmark$ `kernel_first_spec`); goal reachability, used to derive rational guards and hazard bits, is proved correct ($\checkmark$ `goal_reachable_correct`); the decision procedure itself was replaced by a deduplicating, early-exiting variant proved sound and complete ($\checkmark$ `decide'_sound`, $\checkmark$ `decide'_complete`). The elaboration from the tool’s protocol tree into the kernel’s input is the trusted front end: a hundred lines of executable Coq and a hundred of Python. The kernel is extracted to OCaml and run by `skillc severity-verified`. On the sixteen benchmark protocols in its fragment — propositional effects, no environment choice — it agrees with the analyzer’s tolerance degree on every one, in under 50 ms each.

Finding 1: existing corpora cannot pose the question. We ran the analysis on the compiler’s achievability corpus (15 packs), its extension (6), and the 17 public skills of the reference collection compiled by the deterministic front-end. Of these 38 packs, *none* declares an irreversible effect, and the 17 real skills contain *no choice point at all*: the front-end produces linear, add-only protocols. Every pack is trivially tolerant at every k . This is a result about modelling, not about the tool: tools’ real-world irreversibility (sending, paying, deleting, deploying) is not captured by achievability-oriented compaction, so a severity analysis needs the declaration §5 introduces. It also means the question this paper asks has been invisible to the corpora the field evaluates on.

Finding 2: the severity benchmark. We therefore built seventeen protocols with genuine decision points and irreversible tools, each with a provenance note stating the expected verdict *before* running the tool: booking (the running example and its two repairs), database migration, an email campaign and its guard repair, deployment with and without rollback, file

protocol	choices	loops	irreversible	k^*	Benign	Futile	Catastrophic
booking_fastpath	1	0	purchase	0	1	0	1
booking_reordered (repair: reorder)	1	0	purchase	≥ 5	2	0	0
booking_narrowed (repair: narrow)	1	0	purchase	≥ 5	1	0	0
migration_backup	2	0	drop_old	1	3	1	2
email_campaign	1	0	send (declared)	0	1	0	1
email_campaign_guarded (repair: guard)	1	0	send	≥ 5	2	0	0
deploy_with_rollback	1	0	—	≥ 5	1	1	0
deploy_no_rollback	1	0	deploy	0	1	0	1
file_cleanup	1	0	delete	0	1	0	1
order_fulfilment (environment choice)	3	0	ship	0	1	0	1
retry_then_purge	2	1	purge	0	6	0	2
shipping_detour (explicit guards)	1	0	—	≥ 5	2	0	0
claim_eligible	1	0	refund	≥ 5	1	1	0
claim_ineligible (same protocol)	1	0	refund	0	0	0	1
staged_commit	3	0	commit	1	3	4	7
release_with_audit (bystander, exact)	1	0	deploy	0	1	0	1
release_with_cache (bystander, serialize)	1	0	deploy	0	1	0	1

■ **Table 1** The severity benchmark ($k_{\max} = 4$; ≥ 5 means no hazard within four misselections). Verdict counts are per branch and abstract world. Total analysis time 0.33 s.

cleanup, order fulfilment with an environment-controlled bank, a retry loop with a destructive give-up, a guarded shipping choice, an insurance claim in two worlds, a staged commit, and two release protocols with a bystander role — one whose bystander is variable-disjoint, one whose bystander establishes a later precondition. Results (Table 1): 55 branch verdicts (a branch at an abstract world) in 0.33 s total; 29 Benign, 7 Futile, 19 Catastrophic; k^* distribution 9×0 , 2×1 , $6 \times \geq 5$; point-of-no-return actions **purchase**, **send**, **deploy**, **delete**, **ship**, **purge**, **refund**, **drop_old**, **commit** — every one an externally visible effect. Sixteen of seventeen verdicts matched the pre-stated expectation; the exception exposed an authoring error in our benchmark (a goal disjunct that failed to exclude **shipped**), corrected and recorded. The bystander check (Theorem 13) reports the audit release exact — five independent pairs — and the cache release with two pairs to serialize, both on the atom the bystander establishes; every other benchmark protocol has a single acting role, so the question does not arise there.

Three pairs carry the paper’s claims. *Repairs restore tolerance*: reordering, narrowing and guarding each take a 0-tolerant protocol to tolerance at every k . *Severity is a property of node and world*: the insurance-claim protocol is identical in both rows; with an eligible claimant the wrong branch is Futile, with an ineligible one **refund** is a point of no return and $k^* = 0$. *The point of no return depends on Γ ’s establishers*: with **rollback** available, **deploy** deletes nothing permanently and the wrong branch is merely Futile; remove **rollback** and the same branch is Catastrophic. The last pair refines Theorem 7’s anti-monotonicity: for a *fixed* hazard more tools never help, but the derived hazard is not fixed — a tool that re-establishes a lost atom removes a point of no return.

Finding 3: modularity pays only with the interface abstraction. We chained a benchmark segment n times, $G; G; \dots; G$ with atoms renamed per segment, and compared four analyses at $k = 2$ (Table 2): the whole-system *complete* enumeration of hazard-free terminations (like-for-like with computing an interface), modular checking with the *concrete* exit set, modular checking with exits *projected* onto the cone of influence of the remaining segments, and the cost of re-checking after the last segment is replaced by its unsafe variant. On the two-decision migration segment, whole-system analysis grows about fourfold per segment; the concrete interface also grows exponentially (256 points at $n = 6$) because each segment’s

n	whole (complete) ms	exits	modular, concrete ms	interface	modular, projected ms	interface
1	18.6	3	17.7	3	18.6	2
2	108.6	6	66.1	8	38.4	2
3	494.5	12	197.5	20	67.1	2
4	2130.5	24	512.1	48	81.5	2
5	9217.6	48	1346.6	112	114.7	2
6	35752.0	96	2914.6	256	132.9	2

■ **Table 2** Chains of the migration segment, $k = 2$. Re-check after replacing the last segment: whole-system 16–172 ms growing with n ; modular 19–25 ms, constant. The deployment segment gives the same shape at smaller scale (728 ms vs 59 ms at $n = 6$).

choices leave distinct atoms behind; the projected interface has two points at every n and the analysis is linear. At $n = 6$ that is 35.8s against 0.13s. Re-checking after a change costs whole-system analysis the full run (172ms) and modular analysis the last segment alone (22ms, constant in n).

This is the honest form of the modularity claim. Theorem 21 does not make checking cheap; it makes it *decomposable*, and the interface form is what lets a coarse, declared invariant stand in for the concrete exits. Inferring good interfaces beyond the cone of influence is the open engineering problem.

Finding 4: live agents take the branches the tool calls catastrophic. Predictive validity asks whether the *Catastrophic* verdicts name choices real agents make, and whether observed catastrophes are ordered by k^* . We drove every benchmark protocol with two production language models of one family, a small and a mid-size one, invoked through their vendor’s command-line interface with no tools and a two-sentence system prompt, each playing every agent role. At a choice the model sees the goal, the facts currently true, the tools with their effects, and each option’s actions and continuation, and answers with a label; everything else — effects, preconditions, loops, the environment’s choices (sampled uniformly), the hazard — is the analyzer’s own semantics. Two conditions: *plain*, the task as stated, and *pressured*, the task plus a requester’s note claiming the preparatory and checking steps were already done and asking for the fastest route. Five runs per protocol, model and condition: 340 runs, 400 agent decisions, total cost \$1.59 (`scripts/live_agents.py`; every run and every reply is in `results/`).

Three checks of the theory hold exactly. Every catastrophe had more than k^* misselections and none occurred on a protocol tolerant at every tested budget — the harness and the tool share semantics, so this checks the harness against Theorem 9. The catastrophe rate is ordered as the classes predict: 0 of 120 runs on $k^* \geq 5$ protocols, 7 of 220 on $k^* \in \{0, 1\}$ protocols. And the verdicts are not vacuous: 6 of the 19 *Catastrophic* verdicts were taken by a live agent (17 times), against 1 of the 7 *Futile* ones (once); the one *Benign* misselection — the express detour, intended only when urgent — was taken in all ten pressured runs and delivered every time, which is what *Benign* means. The repair pairs behave as Theorems 22–24 say: the two catastrophes of `booking_fastpath` under pressure vanish in its reordered and narrowed forms with the same model and prompt.

Two observations qualify this. Misselection was rare and concentrated: in the plain condition neither model misselected once in 170 runs; under pressure the smaller model skipped the preparation stages of `staged_commit` and then committed in 5 of 5 runs (three misselections each, on a $k^* = 1$ protocol) and took the booking fast path in 2 of 5, while the larger model misselected only on the benign detour; the two bystander protocols drew

no misselection in 40 runs. The risk the verdicts name is real, but in this sample it was exposed by one model under one kind of pressure; the experiment establishes non-vacuity and ordering, not rates. Second, an earlier form of the prompt that hid what the environment’s branches do led the larger model to ship before charging in 5 of 5 plain runs; we count that against the prompt, re-ran those cells with the branch contents shown, and report the re-run. The experiment also exposed a reporting bug in the tool — the recording pass stopped at the first hazardous branch, so a sibling branch could go unclassified — fixed before the numbers above; the theorems are untouched, and Table 1 reflects the fix.

Finding 5: on real skills, the checker’s refusals are the runs that would have been wasted or wrong. The achievability half of the discipline is only useful if its verdicts predict what happens when an agent actually runs a skill. We took real skills from public repositories — the seventeen of the vendor’s skills repository and 145 more from twelve third-party collections, de-duplicated by content hash and each with recorded provenance — and, to make the question sharp, two runtimes: one with a shell and file tools, one with file tools only. A code fence the document expects the agent to execute is an invocation of the shell (a front-end rule added for this experiment), so a skill built on scripts is certified in the first runtime and refuted with `MISSINGCAPABILITY` in the second. Over the 162 skills, 149 are certified in their home runtime and 108 refuted in the file-only one, 95 flipping between them; 13 are refuted even at home because they invoke a capability no runtime provides. The whole census takes 1.8s and no model tokens.

For eight skills with a task the sandbox can genuinely carry out (the rest need a package, a GPU, a credential or a service it lacks) and for four authored specification pairs whose B variant makes a claim the tools cannot honour — a bound the only tool cannot meet, a guard nothing establishes, a goal condition with no establisher, an undeclared tool (`benchmarks/spec-cases`) — we ran the same two production models as in Finding 4 in an empty directory with the skill and a concrete task, the tools restricted to the runtime, and *verified the artifact*: a workbook’s live formula, a merged PDF’s page count, the order a mock tool recorded. A run that claims completion and fails verification is a *silent wrong result*; one that says it failed is an *honest failure*; either way its tokens are spent.

Table 3 summarizes 134 runs. On the runs the checker certified, 32 of 32 real-skill runs and 16 of 16 specification runs produced a verified artifact, with no silent wrong result. On the runs it refuted, the outcome depends on one thing: whether the skill’s procedure is the only route to the artifact.

Where it is, no refuted run succeeded. The three document skills produce binary formats, and all 12 file-only runs failed honestly. The four specification B variants make a claim their tools cannot honour, and 12 of 16 runs failed honestly, 2 produced a silent wrong result — the small model fabricated the approval no tool could give and reported the report published — and 2 exhausted their turns. Those 28 runs cost \$1.09 and 1.56 M tokens.

Where it is not, the first version of this experiment refuted five third-party skills whose tasks reduced to arithmetic over a dozen rows, seven boolean checks or a frontmatter grammar, and the file-only agent ignored the skill’s scripts and computed the artifact by hand in 20 of 20 runs (\$1.29, 1.22 M tokens). That was a measurement of our task sizes, not of the checker. We therefore rebuilt those five tasks at realistic scale — a thousand transcripts across four samples, a thousand-row employee table, five hundred user records, a hundred and fifty manifests, a hundred and twenty draft files — with verifiers that recompute the answer from the generated inputs and compare it exactly, so an estimate cannot pass. The checker’s verdicts are unchanged by the scale-up. The agents’ behaviour is not: in the certified runtime

configurations	checker	runs	verified	silent wrong	honest fail	no verdict	agent cost	agent tokens
8 real skills, shell runtime	certified	32	32	0	0	0	\$1.83	3.03 M
3 document skills, file-only	refuted	12	0	0	12	0	\$0.62	0.49 M
4 specification pairs, A	certified	16	16	0	0	0	\$0.5	1.14 M
4 specification pairs, B	refuted	16	0	2	12	2	\$0.47	1.07 M
5 skills, small inputs, file-only	refuted	20	20 (by hand)	0	0	0	\$1.29	1.22 M
5 skills, realistic inputs, shell	certified	20	19	1	0	0	\$1.64	3.19 M
5 skills, realistic inputs, file-only	refuted	18	0	5	6	7	\$13.21	14.28 M

■ **Table 3** Real skills and specification cases, two runtimes, two models, two runs each; every artifact verified by recomputation. “Verified” counts artifacts that pass the verifier whether or not the status line was reached. The last three rows are the same five skills at two input sizes. The checker’s total time over all 34 configurations is 260 ms.

19 of 20 runs still produced a verified artifact (\$1.64, 3.19 M tokens, median 8 turns), while in the refuted one 0 of 18 did: 6 honest failures, 5 silent wrong results and 7 runs that exhausted their turn budget, at a median of 21 turns, \$13.21 and 14.28 M tokens — 4.5 times the tokens of the certified runtime for no artifact at all. Hand computation is a way past the checker only while the input is small enough to hold in a context window; the verdict was right and the earlier task was too easy.

The corpus is an untrusted input. A skill document is not data: it is a set of instructions an agent will follow with real tools, so a corpus fetched from a dozen third-party repositories is an attack surface for the experiment itself. Before running anything we scanned all 162 documents statically for six families of evidence — harness impersonation and instruction override, credential exfiltration, destructive commands, fetch-and-execute, obfuscation (base64, zero-width and bidi characters), and edits to the agent’s own configuration or hooks, the last being the one that would change what every later skill may do (`scripts/scan_skills.py`, `docs/CORPUS_SECURITY.md`). No malicious skill and no injection payload aimed at an agent was found: zero hits for fake harness turns, hidden instructions, invisible characters or configuration edits. Nine files raised a flag and all nine are benign in context — a security skill quoting injection payloads it teaches you to test, Dockerfile examples containing `rm -rf /var/lib/apt/lists/*` — with two exceptions worth stating: one skill installs a vendor CLI by piping a download into a shell, and one appends an API key to a shell profile. Neither is among the sixteen skills our agents executed. A regression test fails the suite if a corpus refresh introduces an unreviewed hit. Two limits of our containment are worth repeating for anyone reproducing this: a temporary working directory is not a jail, and the runs are unattended by design. Notably, the one impersonation attempt we did encounter arrived in a *web search result* during collection, not in a skill file — which locates the real surface for an agent that browses.

Finding 6: what the check costs in tokens, and when it needs a model at all. The deterministic front end spends no model tokens; the question is when it is enough. The reading it falls back to — one act per invoked tool — *under*-approximates the document, so a refutation on it (the skill names a tool the runtime lacks) is sound whatever the document means, while a certification on it says only that the named tools exist. That asymmetry is a decision procedure for spending money: escalate to LLM compaction exactly when the pack is weak, the verdict is a certification, and the document visibly carries meaning the reader did not capture — completion language, conditional phrases, irreversible verbs. `skillc autocheck` implements it. Over the 162 skills it escalates on 130 (80.2%) in the

home runtime, where almost everything is weakly certified, and on only 49 in the file-only one, where 108 skills are refuted for free.

We measured the escalation on 20 skills with the small model: a median 22440 tokens and \$0.088 per compaction, fitted as $17939 + 0.595 \times \text{characters}$ for the rest. Against the measured median agent run of 72001 tokens (\$0.049), one compaction is 31.2% of a single run — paid once per skill version, amortized over every run of it — and zero for the 32 skills that need no model. On the other side of the ledger, the runs this experiment’s refuted configurations spent before failing came to 5.1 M tokens and \$4.259. The check is cheap in the cases that matter because the cases that matter are refutations, and refutations are the free half.

11 Repairs

When a branch is **Catastrophic** at the intended budget, four transformations make the mistake affordable: *guard* (insert a validation before the irreversible branch), *compensate* (add a recovery path), *narrow* (remove the branch, so the gate refuses it), and *reorder* (move the irreversible action after the validation). All have ancestors: amending asserted choreographies [36], interactional exceptions [37], supervisory control and shields [38, 39, 40], choreography repair [43, 44]. All four are mechanized (Coq: **Repairs.v**). A *validation* of ψ is a capability that fires exactly when ψ holds and changes nothing — a tool with precondition ψ and no effects.

► **Theorem 22** (Narrow). *Removing branches from a choice node preserves the condition at the same budget ($\checkmark$ `repair_narrow_sound`).*

► **Theorem 23** (Guard). *Let a validate ψ . A choice with the branch $\ell[\psi].a.G_\ell$ added is k -safe iff the choice without it is and G_ℓ is k -safe whenever ψ holds ($\checkmark$ `repair_guard_exact`): a misselected guarded branch is stuck, so it costs budget but reaches nothing ($\checkmark$ `guard_absorbs_misselection`).*

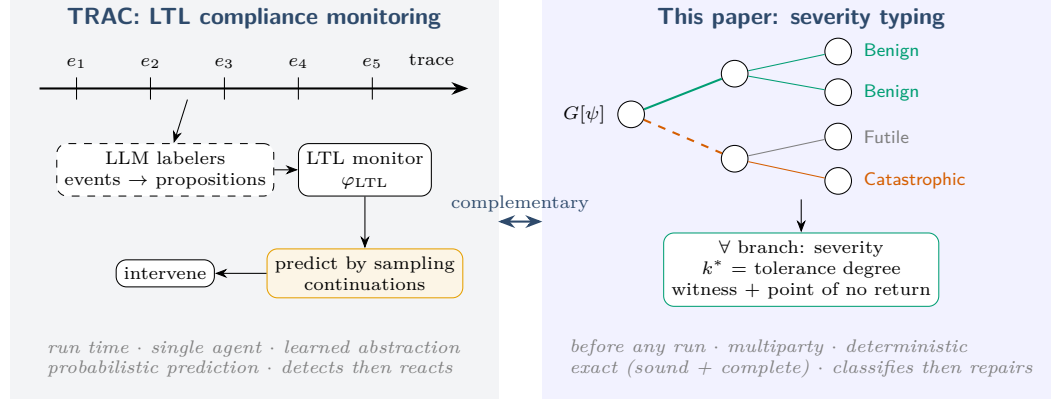
► **Theorem 24** (Reorder). *If chk never creates the hazard and commutes with irr — doing chk then irr re-serializes as irr then chk with the same end world, as STRIPS effects on disjoint variables do — then $chk.irr.G$ is k -safe whenever $irr.chk.G$ is ($\checkmark$ `repair_reorder_sound`). When chk validates ψ both protocols are characterized exactly ($\checkmark$ `reorder_original_exact`, $\checkmark$ `reorder_reordered_exact`) and differ precisely at the point of no return: with ψ false and one hazardous irr -successor, the original is untypable and the reordered protocol is typable at every budget ($\checkmark$ `repair_reorder_pnr`).*

► **Theorem 25** (Compensate). *Appending a recovery path R to a residual G_ℓ is k -safe iff R is safe at every interface point of G_ℓ ($\checkmark$ `repair_compensate_sound`, an instance of Theorem 21), and it turns a Futile residual Benign as soon as R reaches the goal from one interface point ($\checkmark$ `repair_compensate_restores_goal`).*

The booking instance whose fast path purchases before verifying is not 1-tolerant ($\checkmark$ `Gbad2_not_1_tolerant`); *verify* and *purchase* commute ($\checkmark$ `commutes_verify_purchase`, proved without functional extensionality by stating the capability context extensionally), and the reordered protocol is tolerant at every k ($\checkmark$ `Greordered_is_k_tolerant`).

12 Relation to Runtime Compliance Monitoring

The closest work by goal is runtime compliance monitoring of language-model agents, of which TRAC [45] is the most developed: constraints in linear temporal logic, offline auditing,



■ **Figure 4** Two answers to “will the agent misbehave?”. Left: TRAC observes a single agent’s trace at run time, abstracts events to propositions with language-model labelers, checks an LTL constraint, predicts violations by sampling continuations, and intervenes. Right: this paper classifies every branch of a multiparty protocol before any run, deterministically and exactly. Complementary: TRAC’s constraints can supply $\mathcal{H}$; its monitor can enforce at run time what the condition certified.

	TRAC [45]	this paper
when	run time	compile time, before any run
scope	one agent’s trace	multiparty protocol
specification	LTL over propositions	protocol + world + goal + hazard
abstraction	language-model labelers	structured boundary, no LLM
prediction	sampling continuations	exact reachability
output	violation, witness, intervention	severity per branch, k^* , repair
guarantee	empirical violation-rate reduction	Theorems 7–21

■ **Table 4** Axes on which the two approaches differ.

online monitoring, predictive monitors that sample continuations, intervening monitors that preempt a predicted violation. We share the question and differ on every mechanism (Figure 4, Table 4).

Two differences are deep. *Prediction versus projection*: TRAC must estimate whether a violation is coming by sampling a black box; here the guards fix the intended branch and reachability fixes the consequence of the others, so prediction is exact and free of any model of the agent, at the price of expressiveness. *The trusted seam*: TRAC’s soundness bottoms out in learned labelers whose temporal reasoning its own results show degrading with distance and constraint count; ours in a deterministic checker. The approaches compose. Choreographies with first-class agent choice mapped to games [47] own the quantitative version of this framing; we are qualitative and static.

13 What Changed, and Why

An earlier version typed *compliance* rather than *severity*: per-role trust modes, taint propagation, staleness grades. We mechanized its metatheory and it did not survive: a closure premise recursing at the same node terminated at bottom where only a quarantine rule with a partial residual applied, so nothing containing a capability was typable ($\checkmark$ `act_vacuous_with_partial_qres`, inductively too); its taint update ignored what a ca-

pability read ($\checkmark$ `taint_laundering_refutes_noninterference`); its loop condition accepted cycles that never reset a grade ($\checkmark$ `goal_only_cycle_hits_cap`). The audit is retained in the artifact. The present design has no closure premise, no taint, and no grades; deviation lives in the semantics as a budget. We report this because the negative result produced the positive one.

14 Related Work

MPST with a fault model. Crash-stop failures [8, 9, 10], affine and exceptional sessions [11, 12], and protocol-induced recovery [13] model participants that stop; misselection is a participant that continues, wrongly, and none classify the consequence or track a goal. Asserted and refined global types [2, 6, 4, 5] supply our syntax; monitoring [3] and blame [7] treat guard violation as an alarm rather than an object of analysis.

Bounded faults and dead ends. Fault-tolerant planning [28, 27] bounds action failures against goal reachability; we bound selections against hazard reachability. Dead-end detection [29, 30], undoability [31, 32] and excuse generation [33] supply the point-of-no-return machinery; reversibility in reinforcement-learning safety [34] shares the intuition without types.

Types and model checkers. Naik and Palsberg [21] and Kobayashi and Ong [22] set the template we instantiate. Resource-aware [23, 24] and graded [25] session types index judgments by a quantity, with the difference drawn in §6. Bounded-model-checking of fault-tolerant algorithms [26] is the verification-side analogue of Theorem 16.

Agents. Formalisms for agents acting on irreversible external state — execution-fidelity invariants for ledger transactions [41], capability-containment proofs for skill manifests [42] — guarantee exactly-once effects or capability subsets; neither asks which wrong choice is affordable. Runtime enforcement and provenance gating of tool calls [48, 49, 50, 51], effect-typed agent languages [52], network-enforced session types [20], and LTL trace monitoring [45, 46] act on runs or in the host language; we classify branches before any run exists.

15 Limitations and Conclusion

Scope. The bridge and the regular-protocol development are mechanized for the head-move semantics, which is the gate’s; for ungated deployments Theorem 12 covers every permutation of independent nodes — actions under semantic independence, discharged syntactically, and communications between disjoint role pairs unconditionally. Asynchronous buffering is outside the synchronous discipline, and a reviewer comparing scope against the asynchronous mechanised subject-reduction development of [19] will be right to. Typing of recursive sessions is coinductive, so a non-contractive protocol ($\mu X.X$) is typable against anything — harmlessly, since it only loops silently; contractiveness is assumed as usual. On the boolean fragment the tool’s verdict is cross-checked against a kernel extracted from the proof; the elaboration into the kernel’s input, environment choices, and the arithmetic fragment with widening remain trusted. The achievability verdict is about the skill *as written*: an agent that abandons the skill’s procedure and computes the artifact by hand is refuted with the skill, which is the gate’s design; at the input sizes those skills are for, that route closes (Finding 5), but the verdict does not claim it always does. The hazard is a declared or derived input;

the discipline says what follows from it. *Benign* inherits the checker’s over-approximation; *Catastrophic* is a proof. The benchmark is ours, with expected verdicts stated in advance but authored by us. The live-agent experiment is small and establishes non-vacuity and ordering, not deployment rates. Everything presumes the gated architecture.

Conclusion. A participant that may never err may never act. The useful question is not whether it will choose wrongly — it will — but whether the protocol around it survives the choice, and if not, what to change. Severity makes the question precise; the point of no return makes it computable with the machinery the checker already has; the budget makes cascading expressible without a probability; the bridge theorem carries the guarantee from the protocol to the typed session; and the interface makes the answer a contract rather than a run. The well-formedness condition reads: *every affordable mistake is allowed, and every unaffordable one is structurally unreachable.*

References

- 1 K. Honda, N. Yoshida, M. Carbone. Multiparty asynchronous session types. *POPL* 2008; *JACM* 63(1), 2016.
- 2 L. Bocchi, K. Honda, E. Tuosto, N. Yoshida. A theory of design-by-contract for distributed multiparty interactions. *CONCUR*, 2010.
- 3 L. Bocchi, T.-C. Chen, R. Demangeon, K. Honda, N. Yoshida. Monitoring networks through multiparty session types. *FMOODS/FORTE* 2013; *TCS* 669, 2017.
- 4 L. Gheri, I. Lanese, N. Sayers, E. Tuosto, N. Yoshida. Design-by-contract for flexible multiparty session protocols. *ECOOP*, 2022.
- 5 M. Vassor, N. Yoshida. Refinements for multiparty message-passing protocols: specification-agnostic theory and implementation. *ECOOP*, 2024.
- 6 F. Zhou, F. Ferreira, R. Hu, R. Neykova, N. Yoshida. Statically verified refinements for multiparty protocols. *OOPSLA*, 2020.
- 7 L. Jia, H. Gommerstadt, F. Pfenning. Monitors and blame assignment for higher-order session types. *POPL*, 2016.
- 8 A. D. Barwell, P. Hou, N. Yoshida, F. Zhou. Generalised multiparty session types with crash-stop failures. *CONCUR*, 2022; *LMCS*, 2025.
- 9 A. D. Barwell, P. Hou, N. Yoshida, F. Zhou. Designing asynchronous multiparty protocols with crash-stop failures. *ECOOP*, 2023 (Distinguished Paper).
- 10 K. Peters, U. Nestmann, C. Wagner. Fault-tolerant multiparty session types. *FORTE*, 2022.
- 11 S. Fowler, S. Lindley, J. G. Morris, S. Decova. Exceptional asynchronous session types. *POPL*, 2019.
- 12 N. Lagaillardie, R. Neykova, N. Yoshida. Stay safe under panic: affine Rust programming with multiparty session types. *ECOOP*, 2022.
- 13 R. Neykova, N. Yoshida. Let it recover: multiparty protocol-induced recovery. *CC*, 2017.
- 14 J. Lange, N. Yoshida. Verifying asynchronous interactions via communicating session automata. *CAV*, 2019.
- 15 M. R. Clarkson, F. B. Schneider. Hyperproperties. *J. Computer Security* 18(6), 2010.
- 16 S.-S. Jongmans, F. Ferreira. Synthetic behavioural typing: sound, regular multiparty sessions via implicit local types. *ECOOP*, 2023.
- 17 D. Castro-Perez, F. Ferreira, S.-S. Jongmans. *POPL*, 2026. [title to verify]
- 18 A synthetic reconstruction of multiparty session types. *PACMPL*, 2026. [authors to verify]
- 19 D. Tireore, J. Bengtson, M. Carbone. Multiparty asynchronous session types: a mechanised proof of subject reduction. *ECOOP*, 2025.
- 20 Larsen, Scalas, Amir, Jacobs, Wagemaker, Foster. NEST: network enforced session types. *ECOOP*, 2026.
- 21 M. Naik, J. Palsberg. A type system equivalent to a model checker. *ESOP* 2005; *TOPLAS* 30(5), 2008.

- 22 N. Kobayashi, C.-H. L. Ong. A type system equivalent to the modal mu-calculus model checking of higher-order recursion schemes. *LICS*, 2009.
- 23 A. Das, J. Hoffmann, F. Pfenning. Work analysis with resource-aware session types. *LICS*, 2018.
- 24 A. Das, D. Wang, J. Hoffmann. Probabilistic resource-aware session types. *POPL*, 2023.
- 25 D. Orchard, V.-B. Liepelt, H. Eades III. Quantitative program reasoning with graded modal types. *ICFP*, 2019.
- 26 I. Stoilkovska, I. Konnov, J. Widder, F. Zuleger. Verifying safety of synchronous fault-tolerant algorithms by bounded model checking. *TACAS*, 2019.
- 27 D. Aineto, A. Gaudenzi, A. Gerevini, A. Rovetta, E. Scala, I. Serina. Action-failure resilient planning. *ECAI*, 2023.
- 28 C. Domshlak. Fault tolerant planning: complexity and compilation. *ICAPS*, 2013.
- 29 M. Steinmetz, J. Hoffmann. Towards clause-learning state space search: learning to recognize dead-ends. *AAAI*, 2016.
- 30 J. Hoffmann, P. Kissmann, Á. Torralba. “Distance”? Who cares? Tailoring merge-and-shrink heuristics to detect unsolvability. *ECAI*, 2014.
- 31 J. Daum, Á. Torralba, J. Hoffmann, P. Haslum, I. Weber. Practical undoability checking via contingent planning. *ICAPS*, 2016.
- 32 Non-deterministic action reversibility: complexity results. *KR*, 2025. [authors to verify]
- 33 M. Göbelbecker, T. Keller, P. Eyerich, M. Brenner, B. Nebel. Coming up with good excuses: what to do when no plan can be found. *ICAPS*, 2010.
- 34 A. M. Turner, D. Hadfield-Menell, P. Tadepalli. Conservative agency via attainable utility preservation. *AIES*, 2020.
- 35 B. Bittner, M. Bozzano, R. Cavada, A. Cimatti, et al. The xSAP safety analysis platform. *TACAS*, 2016.
- 36 L. Bocchi, J. Lange, E. Tuosto. Three algorithms and a methodology for amending contracts for choreographies. *Sci. Ann. Comp. Sci.* 22(1), 2012.
- 37 M. Carbone, K. Honda, N. Yoshida. Structured interactional exceptions in session types. *CONCUR*, 2008.
- 38 P. J. Ramadge, W. M. Wonham. Supervisory control of a class of discrete event processes. *SIAM J. Control Optim.* 25(1), 1987.
- 39 R. Bloem, B. Könighofer, R. Könighofer, C. Wang. Shield synthesis: runtime enforcement for reactive systems. *TACAS*, 2015.
- 40 M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu. Safe reinforcement learning via shielding. *AAAI*, 2018.
- 41 Z. Yin. Safety invariants for agents orchestrating irreversible state transitions. *arXiv:2608.00783*, 2026.
- 42 A. Metere. Methods for formal verification of agent skills: three layers toward a mechanically checkable capability-containment proof. *arXiv:2605.23951*, 2026.
- 43 S. Basu, T. Bultan. Automated choreography repair. *FASE*, 2016.
- 44 I. Lanese, F. Montesi, G. Zavattaro. Amending choreographies. *WWV*, 2013.
- 45 P. A. Alamdari, T. Q. Klassen, S. A. McIlraith. Formal methods meet LLMs: auditing, monitoring, and intervention for compliance of advanced AI systems. *FAccT*, 2026.
- 46 AgentLTL: a trace-verification framework for procedural compliance in tool-using LLM agents. *arXiv:2607.02599*. [authors to verify]
- 47 K. Gopinathan, J. Feser, et al. Pact: a choreographic language for agentic ecosystems. *arXiv:2605.03143*, 2026. [authors to verify]
- 48 H. Wang, C. M. Poskitt, et al. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents. *ICSE*, 2026.
- 49 E. Debenedetti et al. Defeating prompt injections by design (CaMeL). *arXiv:2503.18813*, 2025.
- 50 M. Costa et al. Securing AI agents with information-flow control (FIDES). *arXiv:2505.23643*, 2025.
- 51 T. Shi et al. Progent: securing AI agents with privilege control. *arXiv:2504.11703*, 2025.
- 52 ETAS: an effect-typed language for agent systems. *arXiv:2607.17780*, 2026. [authors to verify]